

Theory Assignment-1: ADA Winter-2023

Solution (with marking guidelines)

1. Consider the problem of putting L-shaped tiles (L-shaped consisting of three squares) in an $n \times n$ square-board. You can assume that n is a power of 2. Suppose that one square of this board is defective and tiles cannot be put in that square. Also, two L-shaped tiles cannot intersect each other. Describe an algorithm that computes a proper tiling of the board. Justify the running time of your algorithm.

Ans: Tiling always exists: Let $n = 2^r$. First part is to convince that such a tiling always exists. Proof sketch (not for marking) is by induction on r .

Base Case: $r = 1$. Trivially true. Just place the L-shaped tile depending on the location of the defective square.

Induction Hypothesis: The statement holds true for all $1 \leq r \leq k$.

Induction Step: Case $r = k + 1$. Divide the $2^{k+1} \times 2^{k+1}$ square board into 4 equal parts each of size $2^k \times 2^k$. Let the square location (i, j) be the defective one. Without loss of generality, we assume that $i, j \leq 2^r$ (the other cases $i \leq 2^r$ but $2^r + 1 \leq j \leq 2^r$, $2^r \leq i \leq 2^{r+1}$ but $j \leq 2^r$ and $2^r + 1 \leq i, j \leq 2^{r+1}$ are symmetric). It means that the location is in the top-left half. In such case, we put an L-shaped tile with the locations $(2^r + 1, 2^r), (2^r, 2^r + 1), (2^r + 1, 2^r + 1)$. Hence, the other 3 parts also each have one square removed. By induction hypothesis, all the four parts can be properly covered with L-shaped tiles.

Subproblem definition (divide part) - 4 Marks: Divide and Conquer paradigm is the way to solve. The input squareboard is indexed by $(1, 1), \dots, (n, n)$ with $n = 2^r$ with a defective square given in (i, j) . The subproblem is $Tile(A, n, [i, j])$ to put tiles in $n \times n$ board A with (i, j) be the index of the unique defective square.

Divide part is to partition the squareboard into 4 equal parts $A_{\ell t}, A_{\ell b}, A_{rt}$ and A_{rb} as follows.

- $A_{\ell t}$ has indices $(1, 1), \dots, (1, n/2), (2, 1), \dots, (2, n/2), \dots, (n/2, n/2)$, i.e. the top-left half of the squareboard.
- $A_{\ell b}$ has indices $(n/2 + 1, 1), \dots, (n/2 + 1, n/2), (n/2 + 2, 1), \dots, (n, n/2)$, i.e. the bottom-left half of the squareboard.
- A_{rt} has indices $(1, n/2 + 1), \dots, (1, n), (2, n/2 + 1), \dots, (2, n)$, i.e. the top-right half of the board.
- A_{rb} has indices $(n/2 + 1, n/2 + 1), \dots, (n/2 + 1, n), (n/2 + 2, n/2 + 1), \dots, (n, n)$, i.e. the bottom-half of the board.

(4 Marks for specifying each above part)

Putting an L-shaped tile carefully (4 Marks): There are 4 cases.

- Case 1: If the defective square is in the $A_{\ell t}$, i.e. $i, j \leq \frac{n}{2}$, then put an L-shaped tile with the indices $(n/2 + 1, n/2), (n/2, n/2 + 1), (n/2 + 1, n/2 + 1)$.
- Case 2: If the defective square is in the $A_{\ell b}$, i.e. $j \leq \frac{n}{2}$ but $i > n/2$, then put an L-shaped tile with the indices $(n/2, n/2), (n/2, n/2 + 1), (n/2 + 1, n/2 + 1)$.
- Case 3: If the defective square is in the A_{rt} , i.e. $i \leq \frac{n}{2}$ but $j > n/2$, then put an L-shaped tile with the indices $(n/2, n/2), (n/2 + 1, n/2), (n/2 + 1, n/2 + 1)$.
- Case 4: If the defective square is in the A_{rb} , i.e. $i, j > \frac{n}{2}$, then put an L-shaped tile with the indices $(n/2, n/2), (n/2 + 1, n/2), (n/2, n/2 + 1)$.

(1 Mark for each case above)

Combining the solutions (2 Marks): Irrespective of which of the case arises, one square of the chessboard gets removed for each of the parts after putting the L-shaped tiles carefully. So, the recursive algorithm can solve each of the parts separately and combine the solutions directly.

Algorithm Description: (3 Marks)

Let the square board be A with (i, j) be the defective square. We divide the board into 4 equal parts as follows. $A_{\ell t}$ has indices $(1, 1), \dots, (1, n/2), (2, 1), \dots, (2, n/2), \dots, (n/2, n/2)$, i.e. the top-left half of the squareboard. $A_{\ell b}$ has indices $(n/2 + 1, 1), \dots, (n/2 + 1, n/2), (n/2 + 2, 1), \dots, (n, n/2)$, i.e. the bottom-left half of the squareboard. A_{rt} has indices $(1, n/2 + 1), \dots, (1, n), (2, n/2 + 1), \dots, (2, n)$, i.e. the top-right half of the board. A_{rb} has indices $(n/2 + 1, n/2 + 1), \dots, (n/2 + 1, n), (n/2 + 2, n/2 + 1), \dots, (n, n)$, i.e. the bottom-right of the board.

If $i, j \leq \frac{n}{2}$, then put an L-shaped tile with the indices $(n/2 + 1, n/2), (n/2, n/2 + 1), (n/2 + 1, n/2 + 1)$. Recursively call the subroutine for each of the 4 parts with the parameters as follows. $(A_{\ell t}, i, j)$, $(A_{\ell b}, n/2 + 1, n/2)$, $(A_{rt}, n/2, n/2 + 1)$ and $(A_{rb}, n/2 + 1, n/2 + 1)$.

If $j \leq \frac{n}{2}$ but $i > n/2$, then put an L-shaped tile with the indices $(n/2, n/2), (n/2, n/2 + 1), (n/2 + 1, n/2 + 1)$. Recursively call the subroutine for each of the 4 parts with the parameters as follows. $(A_{\ell t}, n/2, n/2)$, $(A_{\ell b}, i, j)$, $(A_{rt}, n/2, n/2 + 1)$ and $(A_{rb}, n/2 + 1, n/2 + 1)$.

If $i \leq \frac{n}{2}$ but $j > n/2$, then put an L-shaped tile with the indices $(n/2, n/2), (n/2 + 1, n/2), (n/2 + 1, n/2 + 1)$. Recursively call the subroutine for each of the parts with the parameters as follows. $(A_{\ell t}, n/2, n/2)$, $(A_{\ell b}, n/2 + 1, n/2)$, (A_{rt}, i, j) and $(A_{rb}, n/2 + 1, n/2 + 1)$.

If $i, j > \frac{n}{2}$, then put an L-shaped tile with the indices $(n/2, n/2), (n/2 + 1, n/2), (n/2, n/2 + 1)$. Recursively call the subroutine for each of the parts with the parameters as follows. $(A_{\ell t}, n/2, n/2)$, $(A_{\ell b}, n/2 + 1, n/2)$, $(A_{rt}, n/2, n/2 + 1)$ and (A_{rb}, i, j) .

Combine the tiling of each of the 4 parts to output a tiling of the entire board.

Running time (2 Marks): When the defective square index (i, j) is given with the board as part of the input, finding out the location of the (i, j) takes $O(1)$ -time. The algorithm divides into 4 subproblems each of size $n/2$. Finally, it combines the solutions in $O(n)$ -time. Hence, the recurrence relation is $T(n) = 4T(n/2) + cn$ with $T(1) = c_1$. Therefore, the running time is $O(n^2)$.

2. Suppose we are given a set L of n line segments in 2D plane. Each line segment has one endpoint on the line $y = 0$, one endpoint on the line $y = 1$ and all the $2n$ points are distinct. Give an algorithm that uses dynamic programming and computes a largest subset of L of which every pair of segments intersect each other. You must also give a justification why your algorithm works correctly.

Ans: The solution sketch with rubric is given as follows. As the input has the segments with coordinates $y = 0$ and $y = 1$, a segment L_i can be represented by the endpoints $(p_1, 0)$ and $(q_1, 1)$ in 2D plane. If the segments are given in the increasing order of x -coordinates with $y = 0$, then we can assume that the segments are $[(p_1, 0), (q_1, 0)], [(p_2, 0), (q_2, 0)], \dots, [(p_n, 0), (q_n, 0)]$ such that $p_1 < p_2 < \dots < p_n$.

Observation: If p_i values are sorted, then for $i < j$, B_i intersects B_j if $q_i > q_j$. Also suppose that B_k intersects B_j such that $j < k$. If L_i intersects L_j with $i < j$, then $q_i > q_j$ and $q_j > q_k$. It means that B_i also intersects B_k . Once they are sorted with respect to p -values, i.e. x -coordinates on $y = 0$ line, then this transitivity of intersection holds. It does not hold true when the segments are in completely arbitrary order.

- **Preprocessing Stage (2 marks):** Sort the segments with respect to the x -coordinates on the $y = 0$ line. (**sorting with respect to the x -coordinates in $y = 1$ line is also correct.**)
- **Subproblem definition (3 marks):** $VALUE(k)$ is the maximum number of line-segments that includes B_k from $B_1, \dots, B_k$ that are pairwise intersecting.
- **Recurrence Relation (4 Marks):** $VALUE(k) = 1 + \max_{1 \leq i < k: B_i \text{ intersects } B_k} VALUE(i)$
(3 Marks)
 If the B_k does not intersect any line segment from $\{B_1, B_2, \dots, B_{k-1}\}$, then $\max_{1 \leq i < k: B_i \text{ intersects } B_k} = 0$.
Base Case: $VALUE(1) = 1$ **(1 Mark).**
- **Subproblem that solves the actual problem (2 Marks):**

$$\max_{1 \leq k \leq n} VALUE(k)$$

- **Algorithm Description (5 Marks):**
 - Sort the line segments with respect to the x -coordinates on the $y = 0$ line. Let $B_1, \dots, B_n$ is the order of segments after this sorting.

- Initialize $Table[1] = 1$.
- For $k = 2, \dots, n$, compute $Table[k]$ as follows. Initialize $Table[k] = 1$. For all $i = 1, \dots, k - 1$, if B_i intersects B_k , then update

$$Table[k] = \max\{Table[k], Table[i] + 1\}$$

- Output $\max_{1 \leq k \leq n} Table[k]$.

Comment: Partially correct but partially incorrect algorithm description deducts 2 marks.

- **Running time (2 Marks):** Computing $Table[k]$ requires $O(k)$ -time and $Table[n]$ is computed only after $Table[k]$ is computed for all $k = 1, \dots, n - 1$. Hence, the running time is $O(n^2)$.

Deduct 1 mark if justification of running time is absent but the answer is correct.

- **Correctness Justification (2 Marks):** The following must be mentioned somewhere in the solution.

Let $B_1 = [(p_1, 0), (q_1, 0)], \dots, B_n = [(p_n, 0), (q_n, 0)]$ are the segments such that $p_1 < \dots < p_n$. It means that the segments are sorted order with respect to x -coordinates in $y = 0$ line. Let B_k intersects B_j such that $j < k$. If B_i intersects B_j with $i < j$, then $q_i > q_j$ and $q_j > q_k$. It means that $q_i > q_k$ with $p_i < p_k$. It means that B_i also intersects B_k . (same as the observation mentioned)

Comment: If somebody gives a justification of optimal substructure property without this explanation then deduct 1 mark.

- Suppose that an equipment manufacturing company manufactures s_i units in the i -th week. Each week's production has to be shipped by the end of that week. Every week, one of the three shipping agents A, B and C are involved in shipping that week's production and they charge in the following:

- Company A charges a rupees per unit.
- Company B charges b rupees per week (irrespective of the number of units), but will only ship for a block of 3 consecutive weeks.
- Company C charges c rupees per unit but returns a reward of d rupees per week, but will not ship for a block of more than 2 consecutive weeks. It means that if s_i unit is shipped in the i -th week through company C , then the cost for i -th week will be $cs_i - d$.

The total cost of the schedule is the total cost to be paid to the agents. Let s_i be the total units of production to be shipped in the i -th week. Then, give an efficient algorithm that computes a schedule of minimum cost. (Hint: use dynamic programming)

Ans: Preprocessing stage is not applicable for this problem. But if company B is used for shipping then it must be used for 3 consecutive weeks. It means that if the last week uses company B , then the last three weeks must use company B . On the other hand,

company C can be used for one week alone. But if it is used for 2 weeks consecutively, i.e. for i -th and $(i + 1)$ -th week, then it can neither be used for the $(i - 1)$ -th week nor be used for the $(i + 2)$ -th week.

- **Subproblem Definition (5 Marks):** We need *three* subproblem definitions.

$CostYC(k)$ is the minimum cost of a schedule for the weeks $\{1, \dots, k\}$ such that the k -th week uses company C (2 Marks).

$CostNC(k)$ is the minimum cost of a schedule for the weeks $\{1, \dots, k\}$ such that k -th week does not use company C (2 Marks).

$Cost(k)$ is the minimum cost of a schedule for the weeks $\{1, \dots, k\}$ (1 Mark).

- **Recurrence Relations (8 Marks):**

$CostYC(k) = \min\{(cs_k - d) + CostNC(k - 1), 2(cs_k - d) + CostNC(k - 2)\}.$

$CostNC(k) = \min\{as_k + Cost(k - 1), 3b + Cost(k - 3)\}.$

$Cost(k) = \min\{CostYC(k), CostNC(k)\}.$

(1 Mark for each of the equations above)

Base Cases: $CostYC(0) = 0, CostNC(0) = 0.$

$CostYC(1) = cs_1 - d$ and $CostNC(1) = as_1.$

$CostNC(2) = as_2 + Cost(1).$

1 Mark for each of the base cases.

- **Subproblem that solves the actual problem (3 Marks):** $Cost(n).$

- **Algorithm Description:** (6 Marks)

– Initialize $TableYC[0] = TableNC[0] = 0, Tab[0] = 0.$

Then, $TableYC[1] = cs_1 - d, TableNC[1] = as_1$ and

$Tab[1] = \min\{TableYC[1], TableNC[1]\}.$

$TableNC[2] = as_2 + Table[1].$

– $TableYC[2] = \min\{TableNC[1] + cs_2 - d, TableNC[0] + c(s_2 + s_1) - 2d\}$ and

$Tab[2] = \min\{TableYC[2], TableNC[2]\}.$

– For $i = 3, \dots, n$, compute

$TableYC[i] = \min\{TableNC[i - 1] + cs_i - d, TableNC[i - 2] + c(s_i + s_{i-1}) - 2d\}$

and $Tab[i] = \min\{TableYC[i], TableNC[i]\}.$

$TableNC[i] = \min\{as_i + Tab[i - 1], 3b + Tab[i - 3]\}.$

$Tab[i] = \min\{TableYC[i], TableNC[i]\}.$

– Output $Tab[n].$

- **Running time (3 Marks):** Initialization part of the algorithm takes $O(1)$ -time. After that three distinct tables are computed each of size n . Computing every entry of a table takes $O(1)$ -time. Hence, the running time is $O(n).$

Acknowledgement: